Voyage-AI — System Architecture & Technical Design Specification

Document Reference: `docs/reports/architecture.pdf` / `docs/reports/architecture.md`
System Version: 0.1.0 (Phase 1 Baseline)
Author: Senior Software Architect & Systems Analyst
Verification Date: October 21, 2026
Status: Implementation-Verified

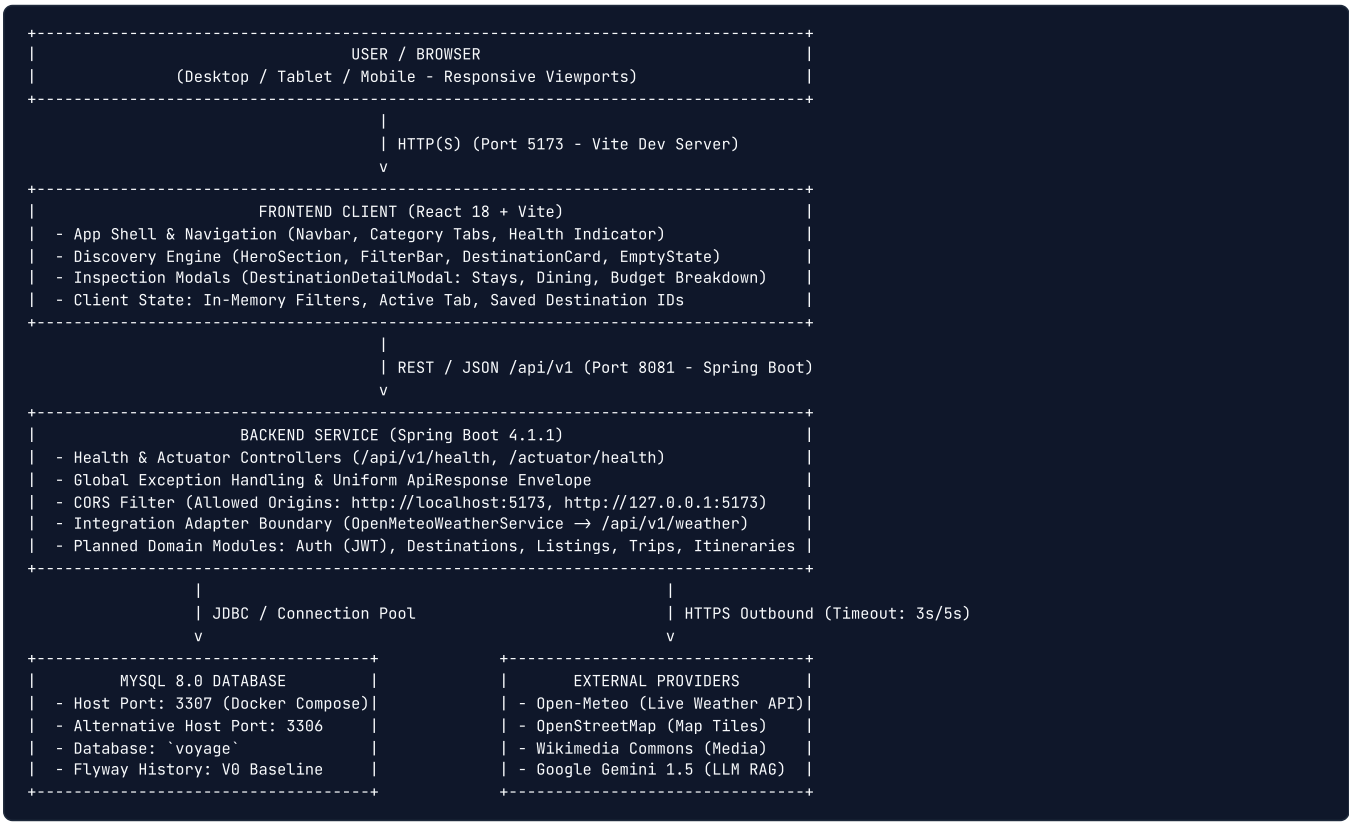
---

1. Executive Architecture Overview

1.1 Project Purpose & System Boundaries

Voyage-AI is an AI-powered travel discovery and itinerary planning platform designed to unify destination exploration, accommodation and dining curation, attraction selection, and day-by-day trip scheduling. The primary architectural objective is to deliver authentic travel intelligence with transparent budget estimation and verifiable data provenance, eliminating the synthetic data and opaque pricing prevalent in commercial aggregators.

The system is architected as a decoupled **modular monolith**:
- **Presentation Boundary (`frontend/`):** A client-side Single Page Application (SPA) built with React 18, TypeScript, and Vite, styled using Tailwind CSS and Emil Kowalski tactile interaction patterns. It provides the user interface for destination browsing, multi-criteria filtering, detail inspection, and trip previewing.
- **Service Boundary (`backend/`):** A Spring Boot 4.1.1 REST application executing on Java 17 LTS. It provides stateless HTTP endpoints rooted at `/api/v1`, global exception mapping, CORS validation, health checks, and isolated provider integration adapters.
- **Persistence Boundary (`infra/` & `backend/src/main/resources/db/migration/`):** A relational storage foundation powered by MySQL 8.0 with version-controlled Flyway schema migrations.
- **Testing & Verification Boundary (`tests/` & module test directories):** Centralized and isolated test suites covering unit logic, integration boundaries, database isolation, and Playwright end-to-end browser automation.



1.2 Core Architectural Decisions & Trade-Offs

1. **Decoupled Modular Monolith vs Microservices:**
2. *Decision:* Structure the backend as a single deployable Spring Boot application with strictly partitioned package boundaries (`common`, `config`, `integration`, and planned `domain` packages) rather than distributed microservices.

3. *Rationale:* Eliminates distributed transaction complexity, network serialization latency, and container orchestration overhead during early product validation, while maintaining clear internal boundaries that enable future service extraction if necessary.
4. **Stateless JWT Authentication with In-Memory Client Storage:**
5. *Decision:* Authenticate protected operations using stateless JSON Web Tokens (JWT) carried in the HTTP `Authorization: Bearer <token>` header, held strictly in React application state memory (never in `localStorage` or accessible cookies).
6. *Rationale:* Mitigates Cross-Site Scripting (XSS) token exfiltration and eliminates Cross-Site Request Forgery (CSRF) vulnerabilities by decoupling session state from browser cookie transport.
7. **Data Provenance & Zero-Fabrication Degradation Mandate:**
8. *Decision:* Every factual travel record and weather forecast must carry explicit provenance metadata (`sourceProvider` , `retrievedAt` , `attribution` , `verificationStatus`). When an external provider is unreachable or a data attribute is unverified, the system emits an explicit `null` or `UNAVAILABLE` state with a truthful user message.
9. *Rationale:* Builds long-term user trust by never presenting synthesized or estimated numbers as confirmed facts.
10. **Port Collision Avoidance on Windows Workstations:**
11. *Decision:* Configure the backend to bind to host port `8081` (defaulting away from port `8080` which is frequently occupied by Windows services such as `AgentService`) and configure Docker Compose to map MySQL container port `3306` to host port `3307` (avoiding collisions with active host `MySQL80` services listening on `3306`).
12. *Rationale:* Enables developer workstations to execute the full stack without port binding conflicts or administrative service termination.

---

2. Verified Technology Stack

The following table documents every technology component verified through direct source-code inspection and runtime execution:

Technology / Component	Verified Version	Architectural Role	Repository Location	Actual Implemented Usage	Verification Evidence
Java JDK	17.0.17 LTS (Oracle)	Backend Runtime & Compiler	`backend/pom.xml` , `tests/pom.xml`	Compiles Spring Boot backend and Java Playwright test framework	`javac -version` , `mvn compile`
Spring Boot	4.1.1	Backend Application Framework	`backend/pom.xml:8`	Core web container, REST controllers, validation, and auto-configuration	`mvn test` , application startup logs
Spring Web MVC	4.1.1	REST API Engine	`backend/pom.xml:27`	Dispatches `/api/v1/health` and `/api/v1/weather` endpoints	`HealthController.java` , `WeatherController.java`
Spring Data JPA / Hibernate	4.1.1 / ORM 6.5+	Data Access Layer	`backend/pom.xml:39`	JPA entity management and relational abstraction (0 entities active in Phase 1)	Hibernate initialization log in `task-121.log`
MySQL Connector/J	8.3.0+ (managed)	Relational Database Driver	`backend/pom.xml:45`	Database connectivity for production and development MySQL 8.0 instances	`application.yml:16`
Flyway Core & MySQL	10.x (managed)	Database Schema Migrations	`backend/pom.xml:52-58`	Applies versioned migrations (`V0__baseline.sql`) and maintains `flyway_schema_history`	`DatabaseFlywayIntegrationTest.java`
H2 Database	2.4.240	Isolated Test Database	`backend/pom.xml:63`	In-memory MySQL compatibility mode for unit and integration tests	`application-test.yml:3` , test logs
Node.js	v24.13.1	Frontend Runtime & Build Tooling	Workstation Host / `frontend/`	Executes Vite development server, Vitest runner, and npm build scripts	`node -v`

Technology / Component	Verified Version	Architectural Role	Repository Location	Actual Implemented Usage	Verification Evidence
React	18.3.1	Client Presentation Library	`frontend/package.json:18`	Declarative UI components, state management, and lifecycle hooks	`frontend/src/App.tsx`, component tree
TypeScript	5.6.2	Frontend Static Typing	`frontend/package.json:38`	Strong typing across components, models, and API response structures	`tsconfig.app.json`, `npm run typecheck`
Vite	5.4.10	Frontend Build Engine & Dev Server	`frontend/package.json:40`	Fast HMR dev server (port 5173) and optimized Rollup production bundler	`vite.config.ts`, `npm run build`
Tailwind CSS	3.4.14	Utility-First Styling Framework	`frontend/package.json:37`	Custom editorial tokens, typography hierarchy, and Emil Kowalski transitions	`tailwind.config.js`, `index.css`
Lucide React	0.453.0	Iconography	`frontend/package.json:17`	Consistent vector icons (Compass, Sparkles, MapPin, Heart, Calendar)	`App.tsx`, `Navbar.tsx`
Vitest	2.1.9	Frontend Unit Test Runner	`frontend/package.json:41`	Component unit tests and DOM assertion execution	`npm run test:unit` (7/7 passed)
React Testing Library	16.0.1	DOM Testing Utilities	`frontend/package.json:25`	User-centric DOM query assertions and keyboard event dispatching	`App.test.tsx`, `ImageWithFallback.test.tsx`
Java Playwright	1.48.0	End-to-End Browser Automation	`tests/pom.xml:30`	Cross-browser UI testing, route interception, and responsive layout audit	`tests/src/test/.../browser/`
Docker Engine & Compose	29.6.2 / v5.3.1	Container Orchestration	`infra/docker-compose.yml`	Local orchestration of MySQL 8.0 container on port 3307	`docker inspect voyage_ai_mysql`

---

3. Component Architecture & Domain Boundaries

3.1 Backend Module Architecture (`com.voyageai`)

The backend is structured into modular packages, isolating infrastructure, configuration, common cross-cutting concerns, and domain adapters:

```
backend/src/main/java/com/voyageai/
├── VoyageAiApplication.java                [Application Entry Point]
├── common/
│   ├── api/
│   │   ├── ApiResponse.java                [Uniform Success Envelope: {success, message, data, timestamp}]
│   │   ├── ErrorResponse.java              [Uniform Error Envelope: {success: false, code, message, timestamp}]
│   │   └── GlobalExceptionHandler.java     [ControllerAdvice: 400, 404, 500, 503 HTTP Mapping]
│   └── health/
│       └── HealthController.java           [GET /api/v1/health Endpoint]
├── config/
│   └── CorsConfig.java                     [WebMvcConfigurer: Origins, Methods, Headers, MaxAge]
└── integration/
    └── weather/
        ├── WeatherProvider.java            [Domain Interface for Meteorological Adapters]
        ├── OpenMeteoWeatherService.java    [RestClient Adapter for Open-Meteo Public API]
        ├── WeatherController.java          [GET /api/v1/weather REST Controller]
        ├── WeatherServiceException.java    [Domain Exception with HTTP Status & Error Codes]
        ├── WmoWeatherCode.java             [WMO Weather Code Interpretation Enum]
        └── dto/
            ├── CurrentWeatherDTO.java      [Temperature, Humidity, Wind, Condition DTO]
            ├── DailyForecastDTO.java       [Daily High/Low Temperature Forecast DTO]
            ├── OpenMeteoApiResponse.java   [Raw Open-Meteo JSON Deserialization Target]
            ├── WeatherProvenanceDTO.java   [Data Provenance, Timestamp, License, Attribution]
            ├── WeatherResponse.java        [Unified Weather Response Model]
            └── WeatherUnitsDTO.java        [Metric Units Definition DTO]
```

Responsibilities & Boundaries:

- **`VoyageAiApplication.java`**: Bootstraps the Spring context, scans configuration, and initializes the embedded Tomcat container on port `8081`.
- **`common.api`**: Enforces standard JSON serialization structures. All successful controller responses wrap payloads in `ApiResponse<T>`, while exceptions are translated into `ErrorResponse` with structured error codes (`MISSING_PARAMETER`, `TYPE_MISMATCH`, `INVALID_ARGUMENT`, `WEATHER_PROVIDER_TIMEOUT`, `WEATHER_PROVIDER_ERROR`, `INTERNAL_SERVER_ERROR`).
- **`config.CorsConfig`**: Intercepts cross-origin preflight requests (`OPTIONS`) and configures permissible headers (`Authorization`, `Content-Type`, `Accept`) and origins (`http://localhost:5173`, `http://127.0.0.1:5173`) with a 3600-second cache duration.
- **`integration.weather`**: Decouples meteorological requests from upstream network transport. `OpenMeteoWeatherService` encapsulates coordinate range validation (`[-90, 90]`, `[-180, 180]`), timeout enforcement (3000ms connect, 5000ms read via `SimpleClientHttpRequestFactory`), and WMO weather code mapping.

3.2 Frontend Component Architecture (`frontend/src`)

The frontend client uses a component hierarchy aligned with the Impeccable Design System:

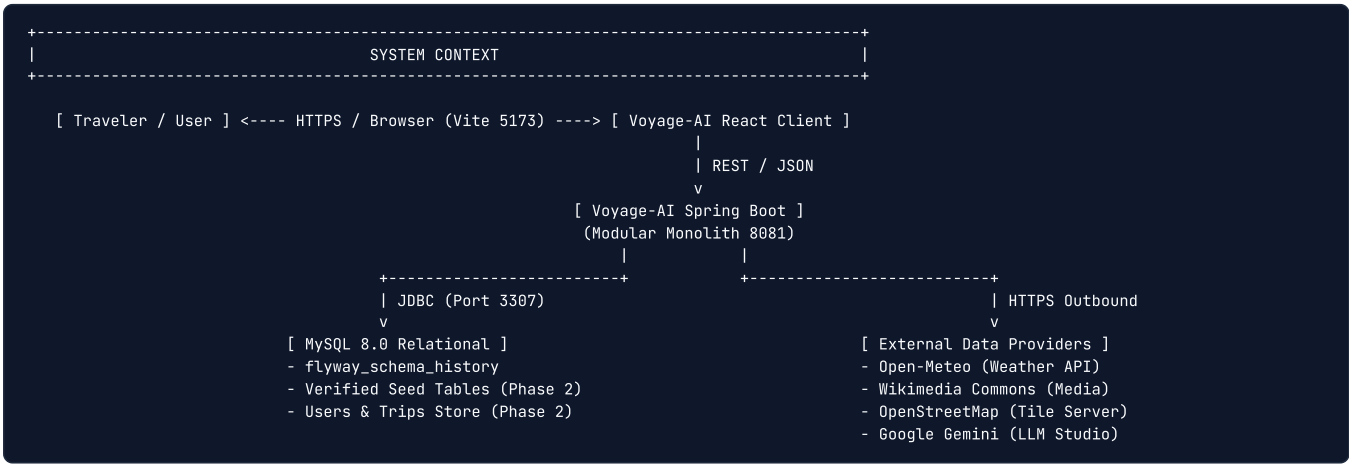
```
frontend/src/
├── main.tsx                              [Client Entry Point & React Root]
├── App.tsx                               [Top-Level Layout, Filter State & Tab Coordinator]
├── index.css                             [Tailwind Base Directives & Custom Button Styles]
├── components/
│   ├── common/
│   │   ├── Badge.tsx                     [Semantic Status & Tag Pills]
│   │   ├── Button.tsx                    [Tactile Button: 5 Variants, :active:scale(0.97)]
│   │   ├── EmptyState.tsx                [Zero-Results Feedback with 1-Click Filter Reset]
│   │   ├── ImageWithFallback.tsx         [Graceful Image Loader with Accessible Fallback Plate]
│   │   └── SkeletonCard.tsx              [Accessible Shimmer Loading Skeletons]
│   ├── layout/
│   │   └── Navbar.tsx                    [Brand Header, Category Tabs, Live Health Dot]
│   └── discovery/
│       ├── DestinationCard.tsx           [Editorial Destination Card with Climate & Budget]
│       ├── DestinationDetailModal.tsx    [5-Tab Inspection Dialog with ESC Listener]
│       ├── FilterBar.tsx                 [Multi-Attribute Region, Style, & Budget Filters]
│       └── HeroSection.tsx               [Warm Modernism Hero Showcase with Value Pillars]
├── data/
│   └── mockDestinations.ts               [Curated Benchmark Data: Jaipur, Varanasi, Munnar, Hampi]
└── types/
    └── destination.ts                    [TypeScript Interfaces for Destinations, Stays, Dining]
```

---

4. Architectural Diagrams

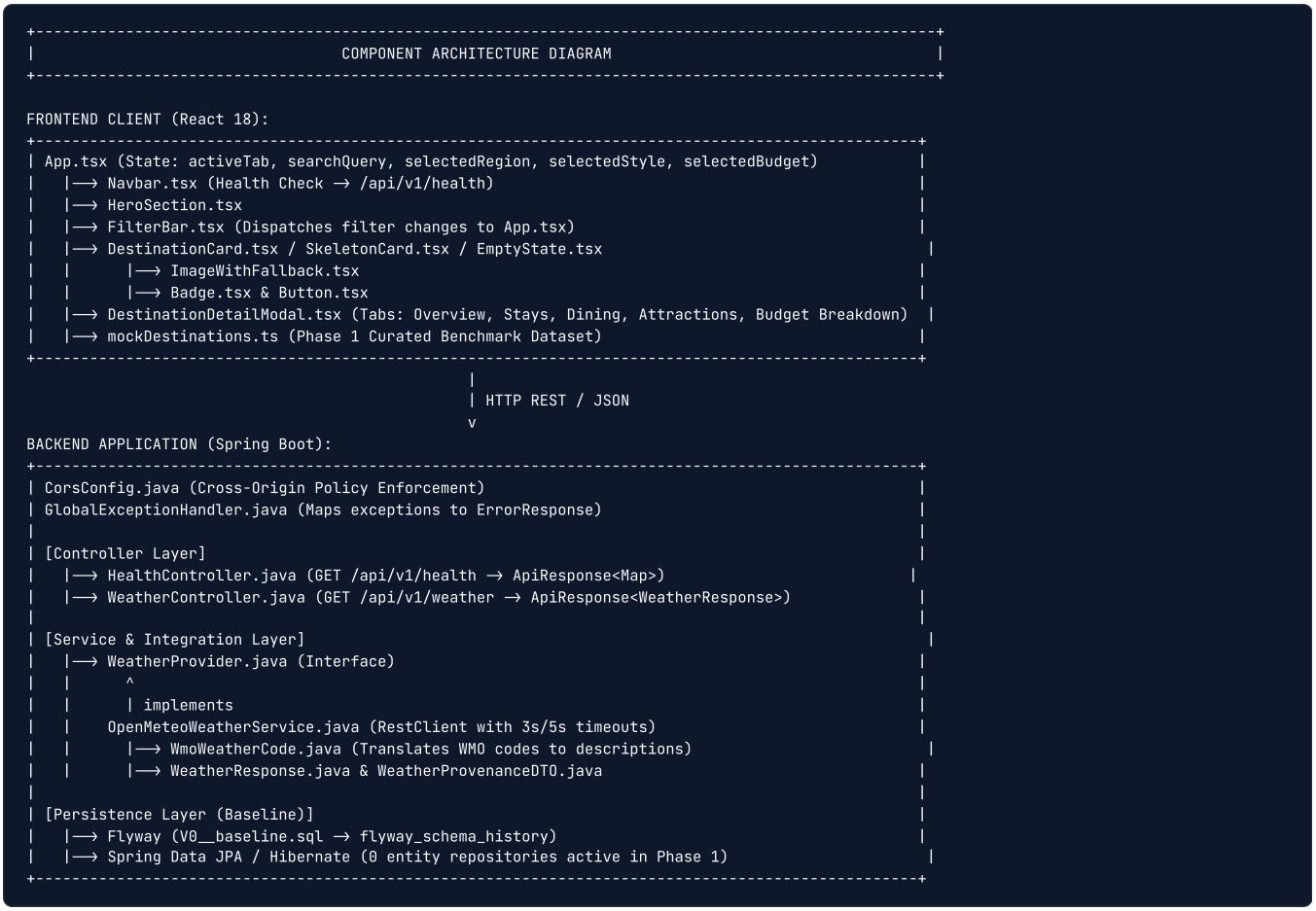
4.1 System Context Diagram

The following diagram illustrates the boundary between Voyage-AI users, the internal subsystems, and external data services:



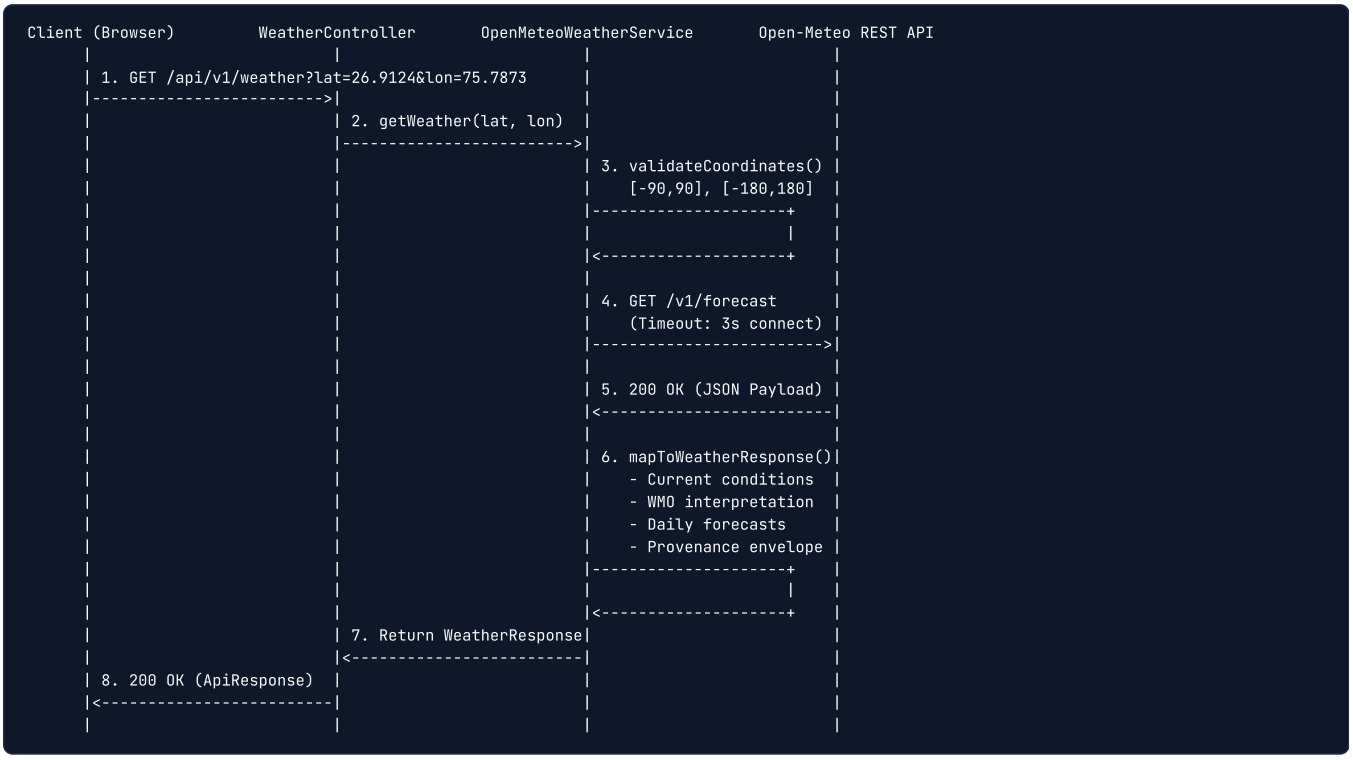
4.2 Component Architecture Diagram

The diagram below details the verified component relationships and dependency flows within the application:



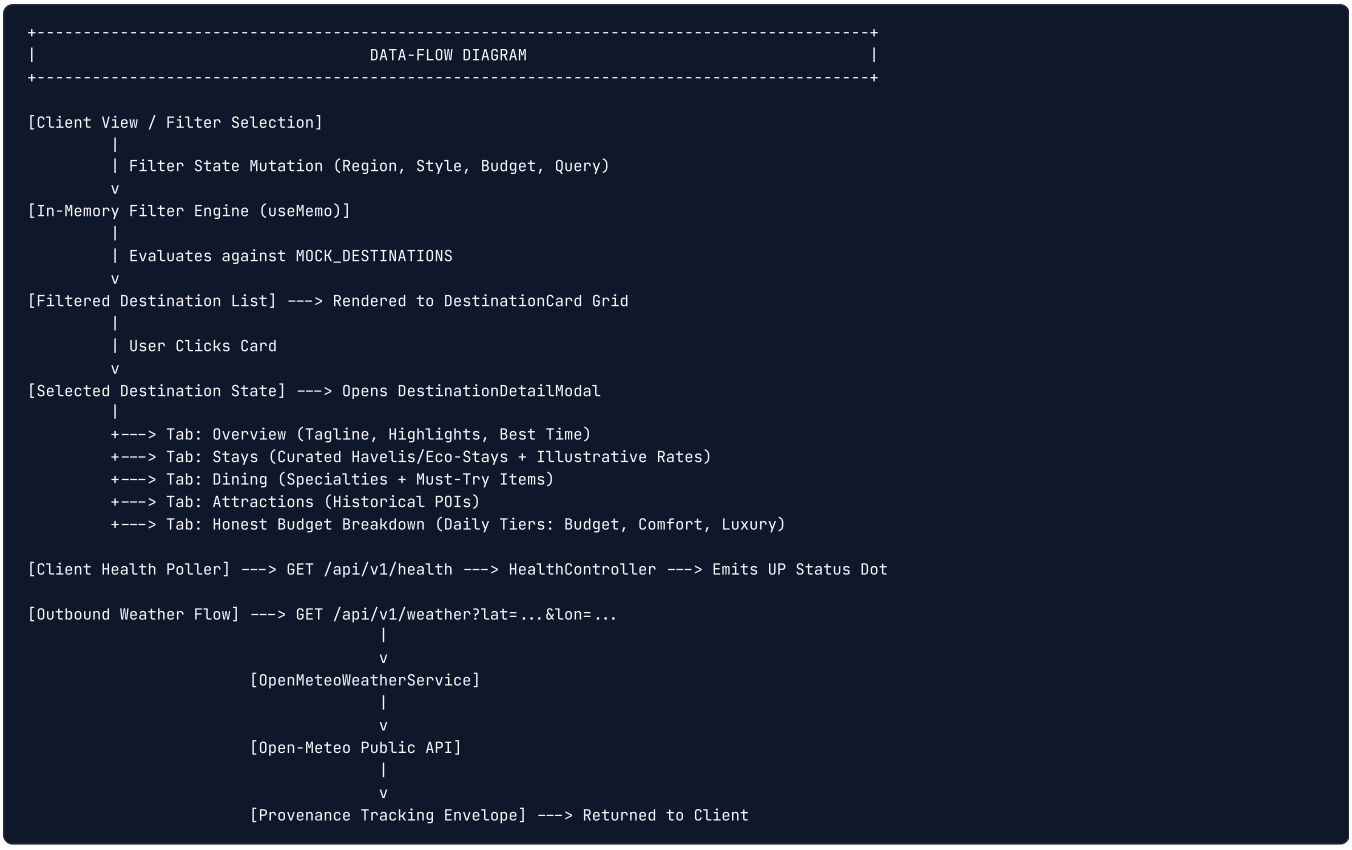
4.3 Runtime Sequence Diagram: Weather Forecast Retrieval Workflow

This sequence diagram illustrates the verified execution path for fetching meteorological data through the backend adapter:



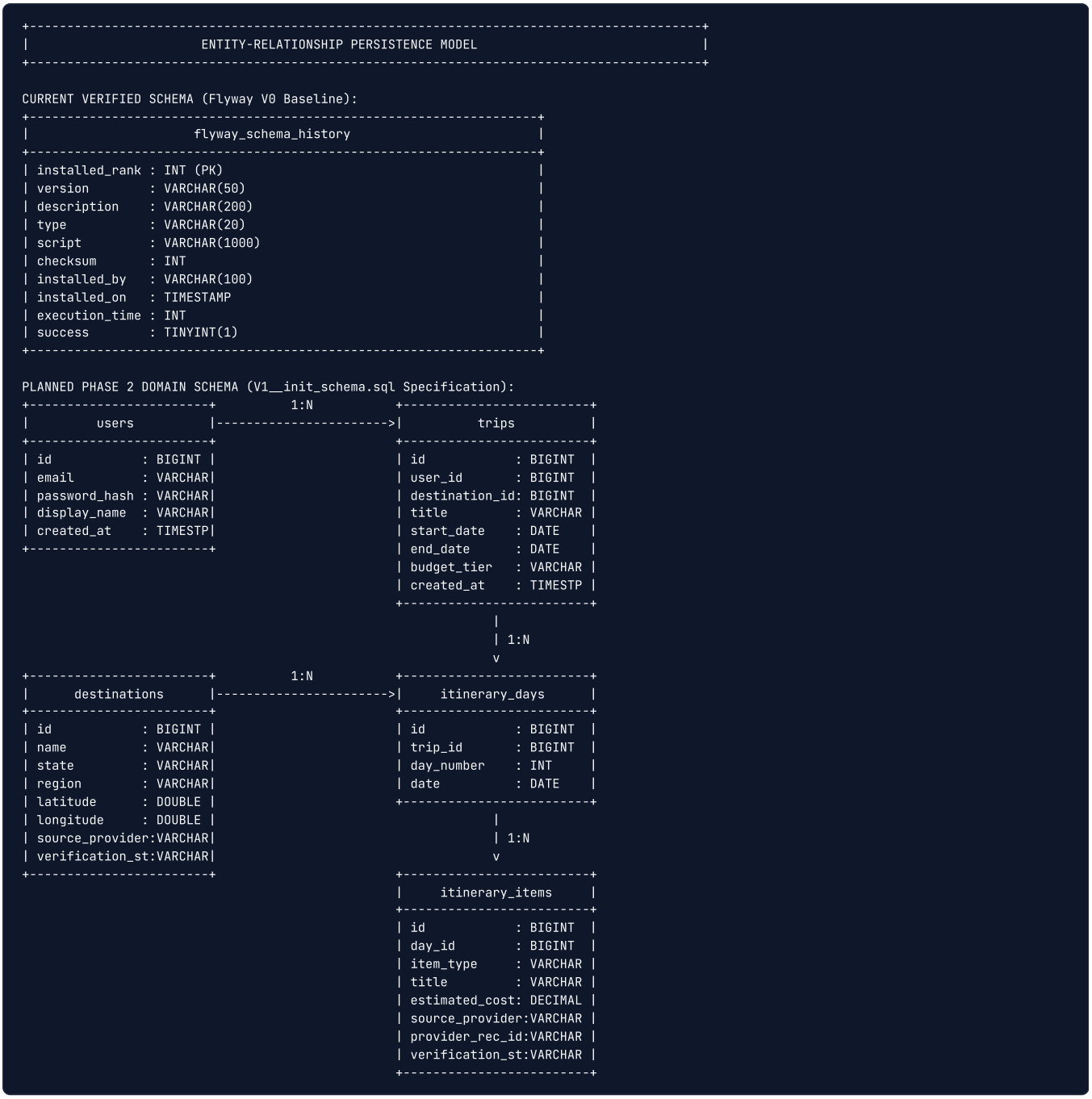
4.4 Data-Flow Diagram

The diagram below traces how travel requests and data move across the presentation, application, and persistence tiers:



4.5 Entity-Relationship Diagram (Database Persistence Model)

The diagram below illustrates the persistence model, distinguishing the **verified Phase 1 Flyway baseline** from the **planned Phase 2 domain schema**:



4.6 Deployment Diagram

The diagram below represents the local deployment targets, port mappings, and process boundaries:

```
+-------------------------------------------------------------------------------------+
|                            LOCAL DEPLOYMENT ARCHITECTURE                            |
+-------------------------------------------------------------------------------------+

[Windows 11 Workstation Host (NT 10.0.26300)]
 |
 ├── [Vite Development Server]
 |    ├── Port: 5173 (`http://localhost:5173`)
 |    └── Serves: React 18 SPA (ESM HMR via Node.js v24.13.1)
 |
 ├── [Spring Boot Process (Maven / Java 17)]
 |    ├── Port: 8081 (`http://localhost:8081`) (Avoids host 8080 collision)
 |    └── Embedded Tomcat, Actuator, Spring Web MVC
 |
 ├── [Containerized MySQL Database (Docker Desktop / WSL2)]
 |    ├── Container: `voyage_ai_mysql` (Image: mysql:8.0)
 |    ├── Host Port Mapping: 3307:3306 (Avoids host 3306 collision)
 |    └── Database: `voyage` (Managed by Flyway)
 |
 └── [Alternative: Native Windows MySQL80 Service]
      ├── Port: 3306 (PID 7716 / Automatic Start)
      └── Database: `voyage`
```

---

5. API & Integration Architecture

5.1 Verified API Endpoints

Endpoint 1: Health Check

- **Route:** `GET /api/v1/health`
- **Controller:** `com.voyageai.common.health.HealthController`
- **Authentication:** Public (No credentials required)
- **Response Structure (200 OK):**
  `json { "success": true, "message": "Service is healthy and responding.", "data": { "status": "UP", "service": "voyage-ai-backend", "profile": "dev", "version": "0.0.1-SNAPSHOT" }, "timestamp": "2026-10-21T18:48:36.551Z" }`
- **Verification Evidence:** `HealthControllerTest.java` (MockMvc), `HealthApiTest.java` (Black-box HTTP).

Endpoint 2: Actuator Health

- **Route:** `GET /actuator/health`
- **Engine:** Spring Boot Actuator
- **Authentication:** Public
- **Response Structure (200 OK):**
  `json { "status": "UP" }`
- **Verification Evidence:** `HealthControllerTest.java`, `application.yml:33`.

Endpoint 3: Weather Forecast Retrieval

- **Route:** `GET /api/v1/weather`
- **Controller:** `com.voyageai.integration.weather.WeatherController`
- **Query Parameters:**
- `latitude` (Double, Required): Value between `-90.0` and `90.0`.
- `longitude` (Double, Required): Value between `-180.0` and `180.0`.
- **Response Structure (200 OK):**
  `json { "success": true, "message": "Weather forecast retrieved successfully.", "data": { "latitude": 26.92, "longitude": 75.82, "timezone": "Asia/Kolkata", "elevation": 435.0, "current": { "time": "2026-10-21T16:30", "temperature": 31.5, "apparentTemperature": 32.0, "relativeHumidity": 42, "windSpeed": 8.5, "precipitation": 0.0, "weatherCode": 0, "weatherCondition": "Clear sky", "isDay": true }, "dailyForecast": [{ "date": "2026-10-21", "weatherCode": 0, "weatherCondition": "Clear sky", "maxTemperature": 34.0, "minTemperature": 21.0 }], "units": { "temperatureUnit": "°C", "apparentTemperatureUnit": "°C", "relativeHumidityUnit": "%", "windSpeedUnit": "km/h", "precipitationUnit": "mm" }, "provenance": { "sourceProvider": "Open-Meteo", "retrievedAt": "2026-10-21T13:18:36.551Z", "attribution": "Weather data by Open-Meteo.com", "license": "CC-BY 4.0", "forecast": true } }, "timestamp": "2026-10-21T18:48:36.551Z" }`
- **Error Responses:**
- `400 Bad Request` with code `MISSING_PARAMETER`: When `latitude` or `longitude` is absent.

- `400 Bad Request` with code `TYPE_MISMATCH`: When parameters are non-numeric.
- `400 Bad Request` with code `INVALID_ARGUMENT`: When coordinates are outside geographic bounds.
- `503 Service Unavailable` with code `WEATHER_PROVIDER_TIMEOUT`: When upstream connection exceeds 3000ms connect or 5000ms read timeout.
- `503 Service Unavailable` with code `WEATHER_PROVIDER_ERROR`: When upstream returns 5xx or 429 rate limit.
- `503 Service Unavailable` with code `WEATHER_DATA_UNAVAILABLE`: When upstream payload is empty.
- **Verification Evidence:** `WeatherControllerTest.java` (7 MockMvc tests), `OpenMeteoWeatherServiceTest.java` (6 MockRestServiceServer tests).

5.2 External Integrations Assessment

Provider	Purpose	Configuration & Auth	Network Constraints	Fallback / Failure Mode	Current Status
Open-Meteo API	Meteorological forecasts and climate indicators	`open-meteo.base-url: https://api.open-meteo.com/v1` (No API key required)	Bounded timeouts (3s connect, 5s read); 10,000 req/day quota	Throws `WeatherServiceException` resulting in 503 HTTP response; zero synthetic temperature generation	**Implemented & Verified**
OpenStreetMap Tiles	Map display via Leaflet.js	Public tile endpoints (Standard OSM tile policy)	HTTP cache headers respected; bulk tile prefetching prohibited	Fallback to static geographic card layout	**Planned (Phase 2)**
Wikimedia Commons	Monument & POI photography	REST API; requires descriptive `User-Agent`	Rate limits per Wikimedia fair use	Local fallback placeholder plate with SVG glyph	**Planned (Phase 2)**
OSRM	Transit distances & drive times	Public demo endpoint or local container	Demo server without SLA; dev evaluation only	Fallback to Haversine direct distance calculation	**Planned (Phase 3)**
Google Gemini 1.5	Itinerary proposal sequencing (RAG)	`gemini.api-key: ${GEMINI_API_KEY}` (AI Studio Free Tier)	15 RPM / 1,500 RPD; strictly zero PII in prompts	Graceful degradation to manual itinerary sequencing	**Planned (Phase 5)**
OpenTripMap	Heritage & attraction database	Candidate requiring API key	1,000 req/day free tier; license evaluation pending	Local curated database seeds	**Unconfirmed Candidate**

---

6. Data & Persistence Architecture

6.1 Database Technologies & Schemas

- **Production Engine:** MySQL 8.0 Community Server.
- **ORM / Abstraction:** Spring Data JPA with Hibernate 6.5+. In the Phase 1 implementation, Hibernate is configured with `ddl-auto: validate` in production/dev profiles, preventing automatic schema alterations.
- **Schema Migration Tool:** Flyway Core 10.x. Migration files reside in `backend/src/main/resources/db/migration/`.
- **Baseline Migration (`V0__baseline.sql`):**
- Executes `SELECT 1;` to confirm database connectivity and initialize the `flyway_schema_history` metadata table.
- Applied and verified against isolated in-memory test databases via `DatabaseFlywayIntegrationTest.java`.

6.2 Data Authenticity & Provenance Tracking Specification

In accordance with the Voyage-AI Data Authenticity Mandate, all persistent factual travel tables introduced in Phase 2 must include non-nullable provenance columns:

```
source_provider     VARCHAR(64)   NOT NULL,
provider_record_id  VARCHAR(128)  NOT NULL,
retrieved_at        TIMESTAMP     NOT NULL,
stale_after         TIMESTAMP     NOT NULL,
license_type        VARCHAR(64)   NOT NULL,
attribution_text    VARCHAR(255)  NOT NULL,
verification_status VARCHAR(32)   NOT NULL DEFAULT 'UNVERIFIED'
```

Rule: The `verification_status` column must default to `'UNVERIFIED'`. It is strictly forbidden to default unverified imported records to `'VERIFIED'`.

---

7. Security Architecture

7.1 Implemented & Designed Controls

Security Domain	Implemented Controls (Phase 1)	Planned Controls (Phase 2 Target)	Status
Authentication	None (public prototype and health checks)	Stateless JWT Bearer tokens signed with HMAC-SHA256 (`HS256`); token stored strictly in React memory	**Designed in `JWT_AUTH_DESIGN.md`**
Authorization	None active	Server-side trip ownership enforcement (`findByIdAndOwnerId`) returning 404/403 on cross-user access	**Designed**
Input Validation	Hibernate Validator (`@NotNull` , `@RequestParam`); coordinate range validation in `OpenMeteoWeatherService`	DTO validation on registration, login, and trip mutation requests	**Implemented & Verified**
Secret Custody	`.env.example` template; `.env` excluded from version control via `.gitignore` ; credentials resolved via environment variables	Production secret management via cloud KMS / environment variables	**Implemented & Verified**
CORS Policy	Restricted to `http://localhost:5173` and `http://127.0.0.1:5173` ; headers limited to `Authorization` , `Content-Type` , `Accept`	Dynamic origin configuration based on deployment profile	**Implemented & Verified**
Transport Security	Local HTTP for development; outbound provider calls enforce HTTPS	TLS 1.3 termination at reverse proxy	**Configured**
Database Isolation	Tests execute against isolated H2 in-memory databases with assertion guards ensuring `voyage` or `pat` databases are never touched	Dedicated least-privilege DB user (`voyage_user`)	**Implemented & Verified**
AI Privacy Protection	No AI prompts executed in Phase 1	Prompts to Gemini free tier strictly scrubbed of all user PII	**Documented Policy**

---

8. Deployment & Runtime Architecture

8.1 Runtime Configuration & Ports

Subsystem	Execution Target	Default Port	Environment Variables	Health Check
Frontend	Vite Dev Server (Node 24)	`5173`	`VITE_API_BASE_URL`	Visual check in browser
Backend	Spring Boot 4.1.1 (Java 17)	`8081`	`SERVER_PORT` , `SPRING_PROFILES_ACTIVE` , `MYSQL_HOST` , `MYSQL_PORT` , `MYSQL_DATABASE` , `MYSQL_USER` , `MYSQL_PASSWORD` , `OPEN_METEO_BASE_URL` , `GEMINI_API_KEY`	`GET /api/v1/health` , `GET /actuator/health`
MySQL (Docker)	Linux Container (`mysql:8.0`)	`3307` (Host) -> `3306` (Internal)	`MYSQL_ROOT_PASSWORD` , `MYSQL_DATABASE` , `MYSQL_USER` , `MYSQL_PASSWORD`	Docker healthcheck (`mysqladmin ping`)
MySQL (Host)	Windows Service `MySQL80`	`3306`	N/A	Windows Service Status (Running)

---

9. Architectural Limitations & Technical Debt

1. **Client-Side Mock Data Decoupling:** In the current Phase 1 implementation, the destination discovery grid (`App.tsx`) queries the static `mockDestinations.ts` data structure rather than calling live backend destination endpoints. Live backend communication is currently restricted to health verification (`/api/v1/health`) and weather queries (`/api/v1/weather`).

2. **Absence of Persistent Domain Entities:** While Flyway connectivity is operational via `V0__baseline.sql`, no domain relational tables (`users`, `destinations`, `trips`) currently exist in MySQL.
3. **Single-Node In-Memory Scalability Bounds:** Caching for weather responses and in-memory JWT blacklists (when implemented) are bounded to single-node memory. Clustered horizontal scaling will require distributed caching (e.g. Redis) in Phase 6.
4. **External Provider Rate Limits:** Open-Meteo (10,000 req/day) and Google Gemini (15 RPM free tier) enforce hard rate ceilings that require aggressive client caching and exponential backoff retry policies.